

ECE 341x Lab 2: Pruning and Weight-only Int8 Quantization with a CUDA Dequant Kernel

Student Lab Manual

Due Date: February 13, 2026

Goal

This two-week lab teaches students how to optimize neural networks for deployment by reducing memory footprint and maximizing hardware utilization. You will learn how to:

- Week 1: Implement unstructured magnitude-based pruning and symmetric per-tensor int8 weight quantization in Python.
- Week 2: Write a custom CUDA kernel to dequantize weights directly on the GPU, effectively bridging the gap between high-level Python logic and low-level hardware performance.

Theory & Background

1. Neural Network Pruning

Pruning is like "weeding" a neural network. It removes weights that don't contribute much to the final answer. In **Unstructured Magnitude Pruning**, we look at the absolute size of every weight. If a weight is very small (close to zero), we assume it isn't important and set it to exactly zero using a boolean mask.

Even though we have fewer "active" weights, standard hardware still treats the tensor as a full block of numbers. You won't see a speedup in this lab just from pruning alone, but you will see how much you can remove before the model starts "forgetting" how to classify images.

2. From Linear to Symmetric Quantization

Quantization is the process of reducing the precision of numbers to save space. Generally, **Linear Quantization** maps a range of floating-point numbers to a range of integers.

Symmetric Per-Tensor Quantization is a specific, simplified version of linear quantization. Instead of using a complex offset (a "zero-point"), we assume the numbers are balanced around zero. By mapping the range $[-a, a]$ to $[-127, 127]$, the "zero" in floating-point stays exactly at "zero" in integer form. This makes the math much faster for the GPU. We calculate a single **scale** for the whole tensor to shrink the values down to 8-bit integers.

3. Under the Hood: The CUDA Execution Model

When we move from Python to CUDA, we are switching from doing things one-by-one to doing thousands of things at once.

- **Grid and Blocks:** The GPU organizes work into a "Grid" made of "Blocks," and each block contains many "Threads."
- **Indexing:** Every thread runs the exact same code. To make sure each thread works on a different piece of the tensor, it calculates its own "ID" or index i using its position in the grid.
- **Memory Traffic:** The biggest bottleneck in AI is moving data from the GPU's memory (VRAM) to its processor. By dequantizing values inside our custom kernel, we move smaller 8-bit integers and only "expand" them to floats right when they are needed, saving precious bandwidth.

The Student Toolbox: Core Methods

Python / PyTorch Functions

- `torch.topk(input, k)`: Grabs the k largest values. Use this to find the "cutoff" point for your pruning mask.
- `torch.abs(input)`: Turns all numbers positive. This helps identify which weights have the most "magnitude."
- `tensor.view(-1)`: Flattens a multi-dimensional weight (like a 3D filter) into one long line of numbers to make math easier.

- `torch.clamp(input, min, max)`: Forces values to stay between a floor and a ceiling. We use this to keep our integers within the 8-bit range of $[-127, 127]$.
- `tensor.numel()`: Tells you the total count of numbers in a tensor.

CUDA C++ / Intrinsics

- `blockIdx, blockDim, threadIdx`: These tell the thread where it is in the GPU "factory" so it knows which weight to process.
- `_float2half_rn(float)`: A fast GPU instruction that converts a standard float to a 16-bit "half" float.
- `data_ptr<T>()`: Gives you the "address" of the data in memory so the C++ code can read it directly.
- `TORCH_CHECK(cond, msg)`: A way to crash the program safely with a helpful message if something goes wrong in the C++ code.
- `cudaGetLastError()`: A "check-engine light" that tells you if the GPU had an error during the last task.

Implementation Pseudocode

1. Magnitude Pruning (`src/prune.py`)

```
# 1. Handle edge cases: if sparsity is 0%, keep
   everything.
# 2. Flatten the weights into a 1D list.
# 3. Use topk to find the value that separates the "big"
   weights from "small" weights.
# 4. Create a mask: True for big weights, False for small
   ones.
```

2. Symmetric Quantization (`src/quant.py`)

```
# 1. Find the biggest absolute value in the tensor.
# 2. Divide that by 127 to get your "scale."
# 3. Divide all weights by this scale, round them, and
   clamp them to  $[-127, 127]$ .
```

3. CUDA Dequantization (cuda_ext/dequant_cuda.cu)

```
// Inside the global kernel:
// 1. Calculate which index 'i' this thread should handle
.
// 2. If the mask at 'i' is True:
//     Multiply the integer weight by the scale and save
//     as a float.
// 3. If False: just save a 0.0.
```

Analyzing Results: Sanity Checks

Use these clues to see if your code is working:

- **Speed:** Your throughput (images per second) might go up as you increase the batch size from 1 to 128.
- **Pruning:** Your model might not get faster when you prune. This is generally expected.
- **Quantization:** The `quant_int8` accuracy should be almost (of course this is arbitrary) identical to the `baseline_fp`. If it drops significantly, check your scale math.
- **CUDA:** Your custom kernel should be slightly more efficient than the Python version because it cuts out the "middle-man" overhead of the Python interpreter.